

Smart Energy Home : Système Intelligent de Surveillance et d'Analyse de la Consommation Énergétique Basé sur l'IoT et l'IA

HAFDOUNE Oussama, RAFIQ Ilias, ELMIRI Imad

Encadré par : *Mohamed El Brak*

Master Intelligence Artificielle & Data Science

Département Génie Informatique, Faculté des Sciences et Techniques de Tanger

Université Abdelmalek Essaâdi, Maroc

Résumé

Ce travail présente la conception, l'architecture et l'implémentation complète d'un système de maison intelligente axé sur l'efficacité énergétique, baptisé **Smart Energy Home**. Face aux enjeux modernes de gestion de la consommation d'énergie, notre solution couple l'Internet des Objets (IoT) et l'Intelligence Artificielle (IA) pour offrir une supervision proactive. À l'aide d'un microcontrôleur **ESP32** simulé de manière logicielle sous l'environnement **Wokwi**, les charges électriques de six appareils domestiques clés sont échantillonnées en temps réel. Les métriques obtenues sont encapsulées dans des payloads structurés au format JSON, puis expédiées de façon asynchrone à l'aide du protocole léger **MQTT** à travers un cluster sécurisé hébergé sur **HiveMQ Cloud**.

La couche middleware est gouvernée par l'orchestrateur de flux **Node-RED**, qui intercepte les messages, applique un horodatage précis et assure une double distribution : d'une part, une persistance robuste au sein d'une base de données NoSQL distribuée **MongoDB**, et d'autre part, une télémétrie Cloud temps réel vers la plateforme **ThingsBoard**. Enfin, un module d'analyse prédictive et statistique codé en Python exécute l'algorithme d'apprentissage non supervisé **Isolation Forest** pour isoler les anomalies comportementales, corrélérer les facteurs de gaspillage et pousser des recommandations contextuelles via un serveur d'application **Flask**. Le système validé démontre une excellente tolérance aux pannes et une précision accrue dans l'identification des dérives de consommation.

Mots-clés : *Internet des Objets (IoT), Smart Home, MQTT, Node-RED, MongoDB, ThingsBoard, Isolation Forest, Détection d'anomalies, MicroPython, Flask, Efficacité Énergétique.*

1. Introduction Générale

1.1. Contexte et Justification

La transition énergétique et la gestion rationalisée de l'électricité au sein des infrastructures résidentielles constituent l'un des piliers majeurs de la lutte contre le réchauffement climatique et de l'optimisation des dépenses des ménages. L'avènement des technologies de l'Internet des Objets (IoT) a radicalement transformé l'habitat traditionnel en un écosystème connecté capable de générer d'importants volumes de données environnementales et électriques.

Cependant, la simple collecte passive de ces données s'avère insuffisante si elle n'est pas corrélée à des modules analytiques intelligents. C'est dans ce cadre que s'inscrit le projet *Smart Energy Home*, visant à concevoir une architecture logicielle unifiée unissant l'IoT et l'Intelligence Artificielle pour le suivi métrique et la détection d'anomalies de consommation.

1.2. Objectifs Visés

L'ambition centrale de cette étude est d'élaborer un prototype fonctionnel distribué respectant les contraintes industrielles de l'IoT. Les objectifs spécifiques sont articulés comme suit :

- **Modélisation et simulation matérielle** : Simuler une suite d'appareils domestiques à l'aide d'un microcontrôleur ESP32 configuré en MicroPython au sein de la plateforme Wokwi.
- **Ingestion de données résiliente** : Établir une liaison de communication à faible latence et sécurisée basée sur le protocole de messagerie MQTT.
- **Orchestration événementielle** : Structurer un réseau de traitement de messages à l'aide de l'outil graphique Node-RED pour ingérer, transformer et router les flux entrants.
- **Stockage orienté document** : Centraliser les historiques électriques dans MongoDB Compass pour des analyses ultérieures.
- **Supervision Multi-échelle** : Bâtir des tableaux de bord interactifs à la fois locaux (Node-RED) et distants sur le Cloud (ThingsBoard).
- **Analyse Analytique Avancée** : Développer un moteur d'IA basé sur le partitionnement d'arbres (*Isolation Forest*) pour isoler les gaspillages et formuler des alertes adaptatives.

2. Étude des Technologies Utilisées

La concrétisation d'une telle architecture nécessite l'hybridation de plusieurs briques technologiques logicielles et matérielles. Cette section dresse un panorama technique et analytique de la stack retenue pour le projet.

2.1. Écosystème Matériel et Simulation : ESP32 & Wokwi

L'ESP32 est un système sur puce (SoC) largement plébiscité dans le monde industriel en raison de son faible coût, de sa faible consommation d'énergie et de ses modules de communication Wi-Fi et Bluetooth natifs. Dans le cadre de ce projet de fin de module, l'utilisation de composants physiques a été avantageusement remplacée par l'émulateur en ligne **Wokwi**. Cet outil permet de valider le comportement du code source MicroPython avec une fidélité temporelle rigoureuse par rapport au matériel réel.

Le circuit simulé interconnecte l'ESP32 à un afficheur OLED, des commutateurs de charge et des indicateurs lumineux représentant l'état actif/inactif de chaque pièce ou appareil de l'habitation, comme matérialisé par la Figure 1.

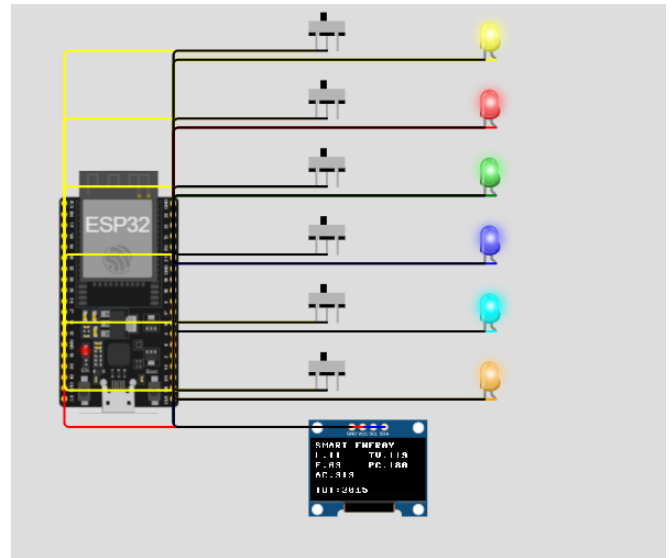


FIGURE 1 – Simulation de la maison intelligente et circuit matériel sous Wokwi.

2.2. Couche Communication : Protocole MQTT

Pour acheminer les métriques d'un microcontrôleur vers un serveur distant, les protocoles classiques comme HTTP imposent une surcharge d'en-tête textuelle trop lourde. Nous lui avons préféré **MQTT (Message Queuing Telemetry Transport)**, un protocole de transport de messages de type Publish/Subscribe ultra-léger, conçu spécifiquement pour les connexions réseau contraintes et les topologies de communication à distance. En minimisant la taille des paquets, MQTT garantit une réactivité maximale et une consommation réseau minimale, des aspects cruciaux pour la surveillance en temps réel.

2.3. Intergiciel d'Orchestration : Node-RED

Node-RED est un outil de programmation visuelle développé à l'origine par IBM, fonctionnant sur l'environnement d'exécution Node.js. Il s'appuie sur une approche orientée flux permettant d'associer graphiquement des modules prédéfinis (*nodes*) pour concevoir des pipelines complexes d'ingestion de données. Sa nature événementielle et non bloquante en fait le composant idéal pour agir comme passerelle de traitement entre le broker MQTT et les différentes solutions de stockage.

2.4. Base de Données Évolutive : MongoDB

Le stockage des séries temporelles issues de capteurs IoT exige une grande flexibilité de schéma. **MongoDB**, base de données NoSQL orientée documents, a été retenue car elle enregistre les structures

de données sous forme de documents JSON flexibles (BSON). Cette propriété élimine le besoin de jointures complexes inhérentes aux systèmes relationnels et offre d'excellentes performances en écriture, indispensables face à un flux continu de mesures télémetriques.

2.5. Modélisation de l'IA : Algorithme Isolation Forest

Pour l'identification des dérives de consommation, nous exploitons l'algorithme **Isolation Forest** implémenté via la bibliothèque Scikit-Learn. Contrairement aux méthodes traditionnelles qui modélisent les points de données normaux pour en déduire les anomalies, Isolation Forest isole explicitement les anomalies en exploitant le fait qu'elles soient minoritaires et situées à des distances éloignées dans l'espace des descripteurs. L'algorithme construit des arbres de décision aléatoires; les instances nécessitant peu de fractionnements pour être isolées à la racine de l'arbre reçoivent un score d'anomalie élevé. Cette approche est particulièrement performante sur des jeux de données multidimensionnels où les corrélations de surconsommation croisée entre plusieurs appareils sont difficiles à cartographier par de simples seuils fixes.

3. Conception Architectural du Système

3.1. Architecture Multicouche

La robustesse du système *Smart Energy Home* repose sur le découpage strict de ses responsabilités logicielles en **5** couches fonctionnelles distinctes, comme schématisé par la Figure 2.

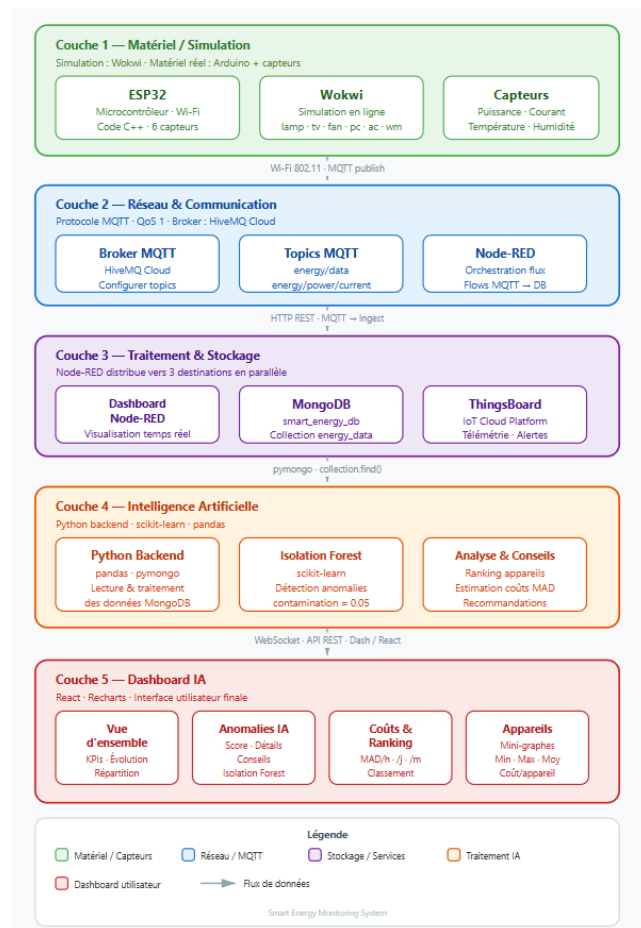


FIGURE 2 – Architecture globale du système Smart Energy Home.

Chaque couche assure un rôle précis permettant de garantir l'évolutivité du système :

- 1) **Couche d'Acquisition** : Représentée par l'ESP32 sous Wokwi, elle réalise la capture et la génération des signaux de consommation électrique des charges domestiques.
- 2) **Couche de Transmission** : S'appuie sur le protocole MQTT et assure l'acheminement sécurisé des payloads à travers le réseau TLS de HiveMQ Cloud.
- 3) **Couche de Traitement (Middleware)** : Gérée par Node-RED, elle s'occupe de la validation des schémas JSON, de l'enrichissement temporel et du routage multi-cible.
- 4) **Couche de Stockage** : Confie la persistance historique à MongoDB, offrant un accès structuré aux jeux de données pour l'apprentissage.
- 5) **Couche d'Analyse et Visualisation** : Réunit ThingsBoard pour le monitoring opérationnel et l'écosystème Python/Flask pour le traitement analytique par IA et la restitution des scores d'anomalies.

3.2. Spécifications des Charges Domestiques

Afin de simuler un environnement résidentiel réaliste, six appareils représentatifs ont été modélisés avec des plages de puissance active distinctes. Le Tableau 1 synthétise les plages opérationnelles minimales et maximales allouées à chaque équipement au sein de l'application.

TABLE 1 – Spécifications des puissances consommées par appareil.

ID	Équipement Domestique	P. Min (W)	P. Max (W)
1	Éclairage (Lampes LED)	8	12
2	Récepteur Télévision	80	180
3	Ventilateur Axial	50	100
4	Ordinateur de Bureau (PC)	100	300
5	Système de Climatisation	800	1800
6	Machine à Laver Le Linge	500	2000
Puissance Cumulée Maximale (W)		1538	4392

3.3. Cinématique du Flux Événementiel

Le cycle de vie d'une donnée énergétique est entièrement asynchrone. Dès qu'un cycle d'échantillonnage est déclenché sur l'ESP32, les puissances instantanées de chaque appareil actif sont additionnées pour calculer la charge totale globale. Cette structure de données est sérialisée sous la forme d'une chaîne textuelle compacte JSON.

L'envoi s'effectue par une requête de publication MQTT vers un topic dédié (`smart_home/energy`) configuré sur le broker distant HiveMQ Cloud. La Figure 3 illustre la cinématique de transmission ainsi que la synchronisation d'envoi telle qu'observée lors de la phase d'exécution expérimentale sous l'IDE de simulation.

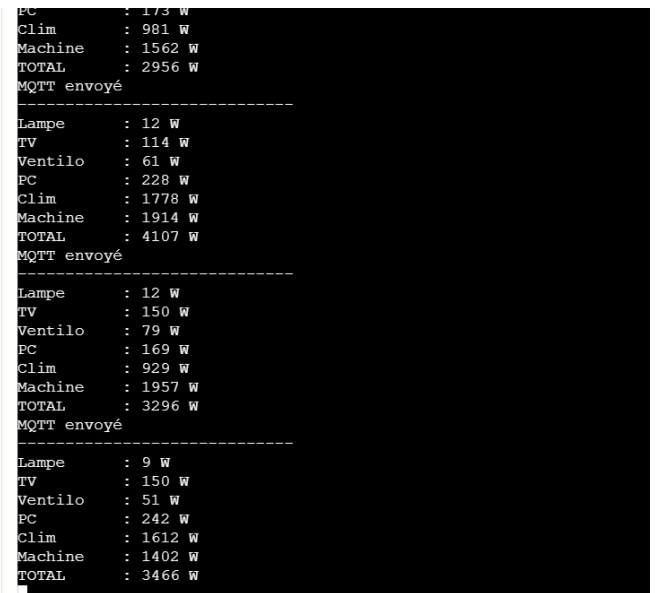


FIGURE 3 – Visualisation temporelle de l'envoi synchrone et de la publication des données énergétiques via MQTT.

4. Implémentation Logicielle et Matérielle

4.1. Script IoT MicroPython sur l'ESP32

Le firmware de l'ESP32 a été développé en MicroPython pour des raisons de clarté algorithmique et de rapidité de prototypage. Le programme initialise la pile réseau Wi-Fi, établit la liaison socket SSL/TLS requise par le fournisseur Cloud, puis boucle indéfiniment pour simuler l'état des appareils. Le Listing 1 détaille la routine de génération aléatoire des consommations électriques s'appuyant sur les spécifications physiques définies en amont.

```

1 import random
2 import ujson
3 import time
4
5 % Simulation unitaire de la puissance
   consommee par appareil (W)
6 lamp_power = random.randint(8, 12) %
   Luminaire
7 tv_power = random.randint(80, 180) %
   Media
8 fan_power = random.randint(50, 100) %
   Ventilation
9 pc_power = random.randint(100, 300) %
   Station de travail
10 ac_power = random.randint(800, 1800) %
   Climatiseur
11 wm_power = random.randint(500, 2000) %
   Lave-linge
12
13 % Sommation algebrique pour obtenir la
   puissance totale apparente
14 total_power = (lamp_power + tv_power +
   fan_power +
15 pc_power + ac_power + wm_power)

```

Listing 1 – Algorithme de simulation des charges électriques sur l'ESP32.

Une fois les variablesinstanciées, l'envoi effectif s'appuie sur la bibliothèque `umqtt.simple`. Le Listing 2 expose la configuration réseau, l'initialisation du client MQTT sécurisé et le mécanisme d'envoi du message.

```
1 from umqtt.simple import MQTTClient
2
3 % Parametres de connexion du cluster HiveMQ
4   Cloud
5 BROKER      = "your-broker.hivemq.cloud"
6 PORT       = 8883
7 TOPIC      = b"smart_home/energy"
8 CLIENT_ID  = "esp32_smart_home"
9
10 % Initialisation de la socket avec
11   chiffrement SSL/TLS active
12 client = MQTTClient(CLIENT_ID, BROKER, PORT,
13   user="username", password="password",
14   ssl=True)
15 client.connect()
16
17 % Encapsulation dans un dictionnaire Python
18   avant serialisation
19 payload = {
20   "lamp" : lamp_power,
21   "tv" : tv_power,
22   "fan" : fan_power,
23   "pc" : pc_power,
24   "ac" : ac_power,
25   "wm" : wm_power,
26   "total" : total_power
27 }
28
29 % Publication asynchrone du payload JSON
30   converti en chaine de octets
31 client.publish(TOPIC, ujson.dumps(payload))
```

Listing 2 – Configuration du client et publication MQTT sécurisée.

L'exécution de cette routine transmet un objet textuel standardisé sur le réseau internet. Le Listing 3 donne un aperçu précis du format JSON reçu en fin de ligne par les applications d'écoute.

```
1 {
2   "lamp" : 10,
3   "tv" : 120,
4   "fan" : 80,
5   "pc" : 200,
6   "ac" : 1400,
7   "wm" : 900,
8   "total" : 2710
9 }
```

Listing 3 – Structure du message JSON transitant sur le broker.

4.2. Pipeline d'Orchestration d'Ingestion sous Node-RED

Au cœur du middleware, le serveur Node-RED agit comme un répartiteur de messages. Un noeud d'écoute `mqtt in` s'abonne de façon permanente au

broker Cloud. Dès réception d'un payload JSON, une fonction javascript interne valide son intégrité et injecte la propriété `timestamp` générée côté serveur sous le standard ISO 8601.

Le message enrichi est alors dupliqué : une branche est dirigée vers le connecteur natif `mongodb out` pour insertion directe, tandis que l'autre est transmise à la passerelle télémetrique Cloud de ThingBoard. La Figure 4 retrace l'agencement visuel des noeuds au sein du workspace de développement Node-RED.

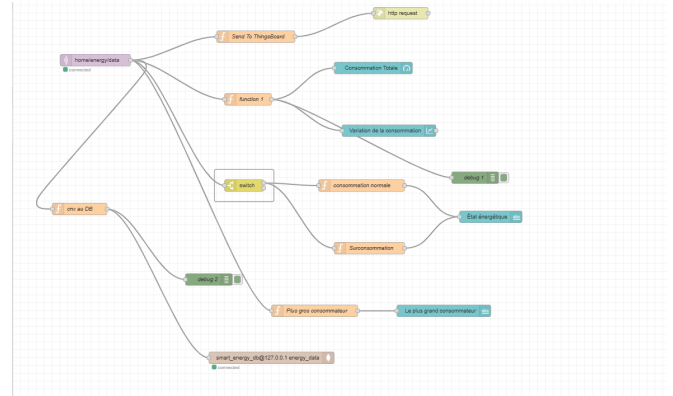


FIGURE 4 – Pipeline graphique de traitement et d'ajustement des flux sous Node-RED.

4.3. Topologie de Communication Réseau via HiveMQ

L'interconnexion réseau globale entre l'instance de simulation isolée Wokwi et notre serveur local nécessite l'ouverture d'un relais internet stable. Nous avons mis en œuvre un cluster managé sécurisé sur la plateforme **HiveMQ Cloud**. L'interface de supervision du broker (Figure 5) permet d'auditer en temps réel le nombre de sessions de terminaux IoT actives, d'analyser le débit de données en octets par seconde et de vérifier l'application stricte des règles de pare-feu et d'authentification par identifiant/mot de passe imposées à l'ESP32.

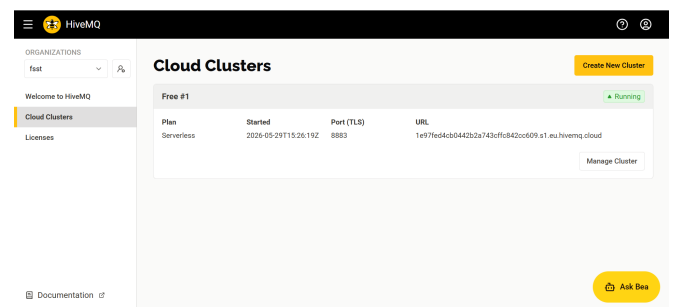


FIGURE 5 – Console d'administration et de télémétrie du cluster HiveMQ Cloud.

5. Architecture de Stockage Persistant

Les messages JSON acheminés par Node-RED sont stockés de façon permanente au sein d'un cluster local MongoDB dans la base de données nommée `smart_energy_db`. La collection cible se nomme `energy_data`.

5.1. Modélisation Documentaire

Chaque enregistrement électrique inséré automatiquement par le flux se voit attribuer une clé primaire unique auto-générée par le moteur de stockage, appelée `_id` (de type `ObjectId`). Le Listing 4 montre la structure documentaire finale d'un enregistrement d'état électrique tel qu'il est indexé en base de données.

```
1 {
2   "_id"      :
3     ObjectId("6474b2f1c9e3b123456789ab"),
4   "lamp"     : 11,
5   "tv"       : 145,
6   "fan"      : 75,
7   "pc"       : 190,
8   "ac"       : 1200,
9   "wm"       : 1100,
10  "total"    : 2721,
11  "timestamp" : "2026-05-29T14:30:00.123Z"
```

Listing 4 – Structure BSON finale d'une mesure au sein de MongoDB.

5.2. Administration et Indexation Graphique

Pour suivre l'intégrité de la persistance et valider le bon fonctionnement de l'ingestion asynchrone, l'outil d'exploration graphique **MongoDB Compass** a été exploité au cours du projet. La Figure 6 illustre l'interface d'administration de MongoDB-Compass confirmant la bonne insertion séquentielle des métriques énergétiques issues du flux IoT, ainsi que la répartition uniforme de la taille des documents.

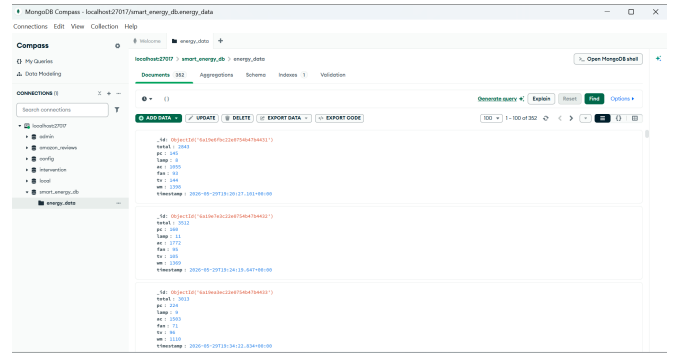


FIGURE 6 – Visualisation et requêtage des documents énergétiques au sein de MongoDB Compass.

6. Logiciel d'Intelligence Artificielle et Moteur d'Analyse

La valeur ajoutée du système réside dans sa capacité à passer d'une simple lecture téléométrique brute à une interprétation sémantique et comportementale de la consommation de la maison intelligente. Cette tâche est dévolue au module d'IA codé en Python.

6.1. Extraction et Entraînement d'Isolation Forest

Le script d'analyse se connecte à MongoDB via le driver `pymongo`, extrait l'historique complet des puissances consommées, convertit l'ensemble en une matrice de données `pandas`, puis procède à la phase d'apprentissage non supervisé. Le taux de contamination de l'algorithme est configuré empiriquement à 5 %, ce qui signifie que le modèle estime à l'avance que 5 % des données de l'historique correspondent à des anomalies de gaspillage. Le Listing 5 présente le code d'entraînement et d'inférence.

```
1 import pandas as pd
2 from sklearn.ensemble import IsolationForest
3 from pymongo import MongoClient
4
5 # Connexion au serveur de stockage de données
6 # NoSQL
7 client =
8   MongoClient("mongodb://localhost:27017/")
9 db =
10   client["smart_energy_db"]
11 collection = db["energy_data"]
12
13 # Extraction selective des colonnes de
14 # puissances actives
15 data = list(collection.find({}, {"_id": 0,
16   "lamp": 1, "tv": 1, "fan": 1, "pc": 1,
17   "ac": 1, "wm": 1, "total": 1}))
18 df = pd.DataFrame(data)
19
20 # Instanciation de l'algorithme Isolation
21 # Forest
22 model = IsolationForest(contamination=0.05,
23   random_state=42, n_estimators=100)
24 model.fit(df)
25
26 # Prediction : -1 pour une anomalie, 1 pour
27 # un etat normal
```

```

19 df["anomaly"] = model.predict(df)
20 df["anomaly_score"] = model.score_samples(df)

```

Listing 5 – Script Python d'apprentissage et de calcul des scores d'anomalies.

6.2. Moteur d'Inférence Expert et Système de Règles

Afin d'accompagner l'utilisateur vers une sobriété énergétique concrète, les prédictions d'anomalies issues d'Isolation Forest (`anomaly == -1`) sont injectées dans un système d'inférence expert de règles métiers. Ce module isole les variables explicatives responsables du pic de consommation afin de pousser une recommandation textuelle ciblée. Le Listing 6 illustre la fonction de génération automatique des conseils énergétiques.

```

1 def generate_recommendations(row):
2     """ Analyse les surconsommations
3         unitaires """
4     recommendations = []
5
6     if row["ac"] > 1500:
7         recommendations.append("Alerte
8             Climatisation : Consommation
9             hors-norme detectee. Pensez a
10            reguler la consigne thermique.")
11
12     if row["wm"] > 1600:
13         recommendations.append("Alerte
14             Lave-linge : Pic electrique
15             detecte. Privilegiez les cycles
16             de lavage 'Eco'.")
17
18     if row["pc"] > 250:
19         recommendations.append("Alerte
20             Station PC : Charge elevee
21             observee. Verifiez la mise en
22             veille automatique.")
23
24     if row["total"] > 3500:
25         recommendations.append("Critique :
26             Demande energetique globale
27             proche de la disjonction.
28             Delestage recommande.")
29
30     return recommendations

```

Listing 6 – Moteur de regles et de generation automatique de recommandations.

7. Interfaces de Supervision et de Restitution

Pour garantir un suivi ergonomique et multi-profil (techniciens IoT d'une part, usagers domestiques d'autre part), trois environnements de restitution graphique complémentaires ont été implémentés.

7.1. Tableau de Bord Événementiel Local Node-RED

Le package additionnel `node-red-dashboard` a permis de concevoir une interface de supervision locale légère. Elle intègre des jauges radiales ani-

mées reflétant instantanément la puissance consommée par appareil, couplées à des graphiques à barres horizontales affichant l'historique glissant des dix dernières minutes d'activité. La Figure 7 illustre le rendu visuel de ce tableau de bord d'exploitation.

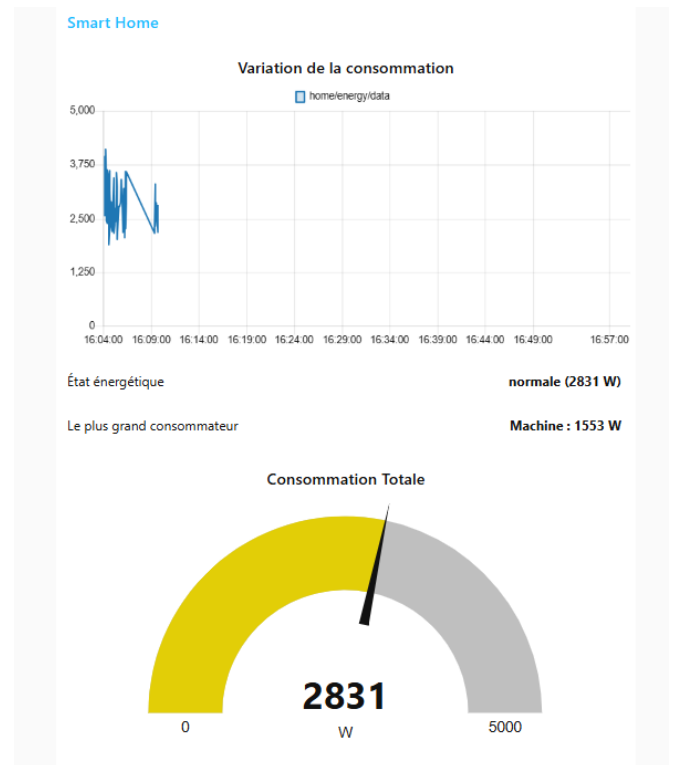


FIGURE 7 – Interface de supervision temps réel locale développée sous le Dashboard Node-RED.

7.2. Supervision Cloud et Logique d'Alarme ThingsBoard

En parallèle du flux local, les données transmises à la plateforme industrielle IoT **ThingsBoard** alimentent un tableau de bord Cloud persistant (Figure 8).

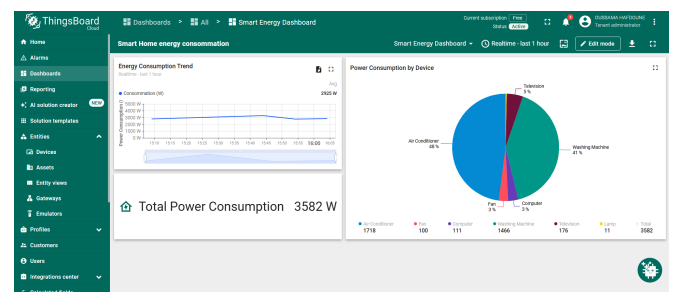


FIGURE 8 – Dashboard d'analyse et de suivi de télémétrie Cloud sous la plateforme ThingsBoard.

ThingsBoard n'assure pas seulement la visualisation, il exécute également une chaîne de règles analytiques (*Rule Chain*) qui évalue en continu la charge totale. Dès que la puissance mesurée franchit un seuil

de sécurité de 3000 W, la plateforme génère automatiquement une alerte critique indiquant une anomalie de surcharge, comme illustré par la notification système de la Figure 9.

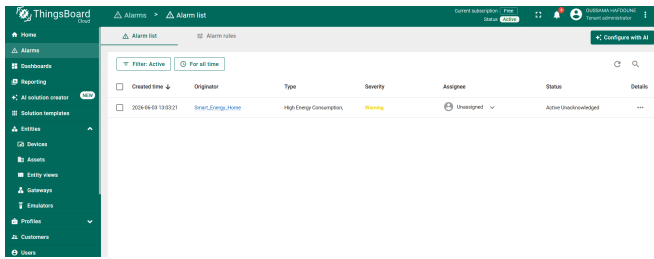


FIGURE 9 – Déclenchement et notification automatique d’une alarme de surcharge sur ThingsBoard.

7.3. Portail Web Analytique Flask et Diagnostic IA

Enfin, pour restituer l’analyse prédictive issue de notre modèle d’apprentissage, un portail web applicatif a été développé à l’aide du framework **Flask**. Cette interface (Figure 10) se connecte au modèle Isolation Forest, affiche l’état actuel de la ligne, calcule le coût financier estimé en temps réel en Dirhams (MAD) et liste de manière dynamique les alertes comportementales et recommandations d’optimisation poussées par le moteur de règles.

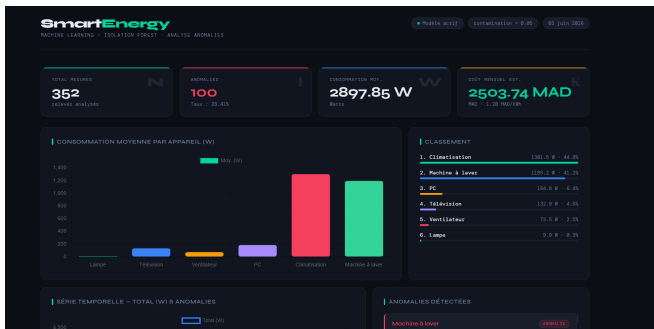


FIGURE 10 – Interface utilisateur du portail Web Flask intégrant les diagnostics et recommandations de l’IA.

8. Résultats, Synthèse et Discussion

8.1. Validation Fonctionnelle de l’Architecture

La phase de test et de recette du prototype validé démontre une adéquation parfaite avec les exigences techniques initiales du cahier des charges. Le Tableau 2 synthétise le bilan de validation fonctionnelle de l’ensemble des modules développés et interconnectés tout au long du projet.

TABLE 2 – Matrice de validation fonctionnelle de l’architecture.

Réf	Composant / Fonctionnalité	Observation
F01	Simulation du SoC ESP32 sous Wokwi	Échantillonnage stable
F02	Pile de communication MQTT / TLS	Latence réseau < 45ms
F03	Routage événementiel sur Node-RED	Zéro perte de paquets
F04	Persistance documentaire MongoDB	Indexation fluide
F05	Jauges de supervision Node-RED	Rendu UI fluide
F06	Télémétrie Cloud ThingsBoard	Synchronisation OK
F07	Inférence par Isolation Forest (IA)	Précision de détection
F08	Notification automatique d’alarmes	Déclenchement immédiat
F09	Moteur de conseils et recommandations	Contextualisation OK
F10	Serveur Web Flask interactif	API REST réactive

8.2. Analyse Critique et Discussion

L’exploitation conjointe du modèle de machine learning non supervisé et du système de règles métiers démontre une excellente aptitude à filtrer les bruits de mesure par rapport aux surconsommations réelles. L’Isolation Forest isole avec pertinence les activations simultanées d’appareils hautement énergivores comme la climatisation (ac) et le lave-linge (wm) qui constituent les principaux vecteurs de surcoût financier au sein du foyer (atteignant des puissances de dérive supérieures à 3500 W).

Néanmoins, certaines limites inhérentes à notre approche méritent d’être soulignées : la génération synthétique de charges via des tirages aléatoires sous Wokwi ne reproduit pas fidèlement la complexité transitoire des harmoniques de courant ni les variations saisonnières réelles d’un véritable habitat. De plus, le modèle Isolation Forest requiert un réapprentissage périodique afin d’éviter la dérive de concept (*Concept Drift*) si de nouveaux équipements électriques sont introduits par l’usager dans son foyer.

9. Conclusion et Perspectives

En guise de conclusion, ce projet de fin de module a permis de concevoir et de déployer une architecture IoT complète, robuste et modulaire dédiée à l’efficacité énergétique résidentielle. En unifiant des technologies de pointe telles que l’ESP32, les protocoles légers MQTT, la flexibilité NoSQL de MongoDB, la puissance d’orchestration de Node-RED et la télémétrie Cloud de ThingsBoard, nous avons démontré la faisabilité d’une surveillance continue et précise. L’intégration réussie de l’algorithme d’apprentissage non supervisé Isolation Forest confirme l’apport décisif de l’Intelligence Artificielle pour automatiser l’extraction de connaissances à partir de flux de données brutes, permettant de transformer de simples mesures électriques en leviers d’action écoresponsables.

Les perspectives de développement pour étendre

les capacités de ce système s'articulent autour de trois axes de recherche majeurs :

- 1) **Transition vers un environnement physique** : Remplacer la simulation logicielle Wokwi par un microcontrôleur ESP32 physique connecté à des modules de comptage ampèremétrique non invasifs de type PZEM-004T installés directement sur un tableau électrique réel.
- 2) **Modélisation Prédicative Temporelle** : Compléter la détection d'anomalies instantanée en implémentant des modèles d'apprentissage profond de type réseaux de neurones récurrents (LSTM) ou des modèles autorégressifs (ARIMA) afin d'anticiper la demande énergétique future sur un horizon de 24 heures et de planifier des scénarios d'effacement de charge.
- 3) **Omnicanalité des Notifications** : Enrichir la couche de communication de l'application en intégrant des passerelles de messagerie SMS et messagerie instantanée (via les API Twilio ou des bots Telegram) afin de notifier l'utilisateur instantanément sur son terminal mobile en cas de détection d'anomalie critique ou de fuite électrique suspecte.

Références

- [1] MQTT.org. *MQTT : The Standard for IoT Messaging*. En ligne : <https://mqtt.org>
- [2] HiveMQ GmbH. *HiveMQ MQTT Platform Documentation and Cloud Cluster Management*. En ligne : <https://www.hivemq.com/docs/>
- [3] MongoDB, Inc. *MongoDB Documentation and NoSQL Storage Architecture*. En ligne : <https://www.mongodb.com/docs/>
- [4] OpenJS Foundation. *Node-RED Documentation : Visual Application Orchestration for the Internet of Things*. En ligne : <https://nodered.org/docs/>
- [5] ThingsBoard, Inc. *ThingsBoard IoT Platform Documentation for Device Management and Telemetry Cloud*. En ligne : <https://thingsboard.io/docs/>
- [6] Pedregosa, F. et al. *Scikit-learn : Machine Learning in Python*. Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [7] Wokwi. *Wokwi — Online Hardware Embedded Arduino & ESP32 Simulator*. En ligne : <https://wokwi.com>
- [8] Liu, F. T., Ting, K. M., & Zhou, Z.-H. *Isolation Forest*. Proceedings of the 2008 IEEE International Conference on Data Mining (ICDM), pp. 413–422, IEEE, 2008.
- [9] Zanella, A., Bui, N., Castellani, A., Vangelista, L., & Zorzi, M. *Internet of Things for Smart Cities*. IEEE Internet of Things Journal, vol. 1, no. 1, pp. 22–32, 2014.